

EXERCÍCIOS PRÁTICOS E TESTES

Material completo para avaliação de aprendizado

PARTE 1: EXERCÍCIOS PROGRESSIVOS

EXERCÍCIO 1: BÁSICO - Sistema de Logging (2-3 horas)

Objetivo: Implementar Simple Factory e Factory Method para diferentes destinos de log.

Requisitos:

1. Interface `ILogger` com método `Log(string level, string message)`
2. Implementações: `ConsoleLogger`, `FileLogger`, `DatabaseLogger`
3. Simple Factory para criação básica
4. Refatorar para Factory Method com classe `LoggingService`
5. Integrar com `IServiceCollection`
6. Criar 5+ testes unitários com xUnit

Critérios de Avaliação:

- Implementação correta de interfaces (20 pontos)
- Factory funcional com validação de entrada (20 pontos)
- Integração com DI adequada (20 pontos)
- Testes cobrindo casos de sucesso e erro (20 pontos)
- Código limpo e bem comentado (20 pontos)

Total: 100 pontos **Tempo estimado:** 2-3 horas **Dificuldade:** ★ Básico

EXERCÍCIO 2: INTERMEDIÁRIO - Sistema de Exportação (4-5 horas)

Objetivo: Implementar Abstract Factory para exportar dados em múltiplos formatos.

Requisitos:

1. Interface `IExporter` com métodos `ExportHeader()`, `ExportData()`, `ExportFooter()`
2. Três famílias: JSON, XML, CSV
3. Cada família tem componentes específicos (Header, Data, Footer)

4. Abstract Factory (`IExportFactory`) com três implementações concretas
5. Classe (`DataExporter`) que usa a factory
6. Permitir troca de formato em runtime
7. Integração completa com DI
8. 10+ testes unitários e 2 testes de integração

Critérios de Avaliação:

- Estrutura correta do Abstract Factory (25 pontos)
- Famílias de produtos consistentes (20 pontos)
- Cliente usa apenas interfaces abstratas (15 pontos)
- Integração DI com lifetimes adequados (15 pontos)
- Testes abrangentes (15 pontos)
- Documentação e exemplos de uso (10 pontos)

Total: 100 pontos **Tempo estimado:** 4-5 horas **Dificuldade:** ★★ Intermediário

EXERCÍCIO 3: AVANÇADO - Sistema Modular de E-commerce (8-10 horas)

Objetivo: Implementar Project Factory completo com módulos interdependentes.

Requisitos:

Módulos obrigatórios:

1. **CatalogModule:** Gerenciamento de produtos
2. **CartModule:** Carrinho de compras (depende de Catalog)
3. **PaymentModule:** Processamento de pagamentos (depende de Cart)
4. **NotificationModule:** Notificações (depende de Payment)

Infraestrutura:

- Interface (`IModule`) com metadados
- Atributo (`[Module]`) para dependências
- (`ProjectFactory`) com resolução topológica
- Descoberta automática via reflexão
- Registro e inicialização em ordem correta

Funcionalidades:

- Adicionar produto ao catálogo
- Adicionar produto ao carrinho
- Processar pagamento (escolher método)
- Enviar notificação de confirmação
- Permitir desabilitar módulos opcionais

Qualidade:

- 20+ testes unitários
- 5+ testes de integração
- Documentação completa (README)
- Diagramas de classe e sequência
- Apresentação em slides (10-15 slides)

Critérios de Avaliação:

- Arquitetura modular bem definida (20 pontos)
- Resolução correta de dependências (20 pontos)
- Project Factory funcional e extensível (15 pontos)
- Integração DI avançada (15 pontos)
- Testes abrangentes e bem estruturados (10 pontos)
- Documentação e diagramas (10 pontos)
- Apresentação e defesa oral (10 pontos)

Total: 100 pontos **Tempo estimado:** 8-10 horas **Dificuldade:** ★★☆☆ Avançado

PARTE 2: TEMPLATES DE TESTES

Template 1: Teste Básico de Factory

csharp

```
using Xunit;
using Microsoft.Extensions.DependencyInjection;

namespace FactoryTests
{
    public class BasicFactoryTests
    {
        private IServiceProvider CreateServiceProvider()
        {
            var services = new ServiceCollection();
            // Registre seus serviços aqui
            services.AddTransient<ConcreteProduct>();
            services.AddSingleton<IFactory, Factory>();
            return services.BuildServiceProvider();
        }

        [Fact]
        public void Create_WithValidType_ReturnsCorrectInstance()
        {
            // Arrange
            var provider = CreateServiceProvider();
            var factory = provider.GetRequiredService<IFactory>();

            // Act
            var product = factory.Create("validType");

            // Assert
            Assert.NotNull(product);
            Assert.IsType<ConcreteProduct>(product);
        }

        [Theory]
        [InlineData(null)]
        [InlineData("")]
        [InlineData("invalid")]
        public void Create_WithInvalidType_ThrowsException(string type)
        {
            // Arrange
            var provider = CreateServiceProvider();
            var factory = provider.GetRequiredService<IFactory>();

            // Act & Assert
            Assert.ThrowsAny<Exception>(() => factory.Create(type));
        }

        [Fact]
```

```
public void Create_CalledMultipleTimes_ReturnsNewInstances()
{
    // Arrange
    var provider = CreateServiceProvider();
    var factory = provider.GetRequiredService<IFactory>();

    // Act
    var product1 = factory.Create("type");
    var product2 = factory.Create("type");

    // Assert
    Assert.NotSame(product1, product2);
}
}
```

Template 2: Teste de Abstract Factory

csharp

```

public class AbstractFactoryTests
{
    [Fact]
    public void Factory_CreatesConsistentFamily()
    {
        // Arrange
        var factory = new ConcreteFactory1();

        // Act
        var productA = factory.CreateProductA();
        var productB = factory.CreateProductB();

        // Assert
        Assert.IsType<ConcreteProductA1>(productA);
        Assert.IsType<ConcreteProductB1>(productB);
        // Verificar que produtos são da mesma família
    }

    [Fact]
    public void Client_WorksWithAnyFactory()
    {
        // Arrange & Act & Assert para Factory1
        var factory1 = new ConcreteFactory1();
        var client1 = new Client(factory1);
        var result1 = client1.Execute();
        Assert.NotNull(result1);

        // Arrange & Act & Assert para Factory2
        var factory2 = new ConcreteFactory2();
        var client2 = new Client(factory2);
        var result2 = client2.Execute();
        Assert.NotNull(result2);
    }
}

```

Template 3: Teste de Integração com DI

csharp

```
public class DIIntegrationTests : IDisposable
{
    private readonly ServiceProvider _serviceProvider;

    public DIIntegrationTests()
    {
        var services = new ServiceCollection();

        // Configurar DI completo
        services.AddLogging();
        services.AddTransient<ProductA>();
        services.AddTransient<ProductB>();
        services.AddSingleton<IFactory, Factory>();
        services.AddScoped<ServiceThatUsesFactory>();

        _serviceProvider = services.BuildServiceProvider();
    }

    [Fact]
    public void Service_UsesFactory_Successfully()
    {
        // Arrange
        var service = _serviceProvider.GetRequiredService<ServiceThatUsesFactory>();

        // Act
        var result = service.DoWork();

        // Assert
        Assert.True(result.Success);
    }

    [Fact]
    public void Factory_ResolvesDependencies_Correctly()
    {
        // Arrange
        var factory = _serviceProvider.GetRequiredService<IFactory>();

        // Act
        var product = factory.Create("type");

        // Assert
        Assert.NotNull(product);
        // Verificar que dependências do produto foram injetadas
    }

    public void Dispose()
    {
    }
}
```

```
{  
    _serviceProvider?.Dispose();  
}  
}
```

Template 4: Teste de Project Factory

csharp


```

public class ProjectFactoryTests
{
    [Fact]
    public void BuildProject_LoadsAllModules()
    {
        // Arrange
        var services = new ServiceCollection();
        var mockRegistry = new Mock<IModuleRegistry>();
        var mockLoader = new Mock<IModuleLoader>();

        var modules = new List<IModule>
        {
            new TestModule1(),
            new TestModule2()
        };

        mockLoader.Setup(l => l.DiscoverModules()).Returns(modules);

        var factory = new ProjectFactory(mockLoader.Object, mockRegistry.Object);

        // Act
        factory.BuildProject(services);

        // Assert
        mockRegistry.Verify(r => r.Register(It.IsAny<IModule>()), Times.Exactly(2));
    }

    [Fact]
    public void BuildProject_RespectsModuleDependencies()
    {
        // Arrange
        var loadOrder = new List<string>();

        var moduleB = new Mock<IModule>();
        moduleB.Setup(m => m.Name).Returns("ModuleB");
        moduleB.Setup(m => m.ConfigureServices(It.IsAny<IServiceCollection>()))
            .Callback(() => loadOrder.Add("ModuleB"));

        var moduleA = new Mock<IModule>();
        moduleA.Setup(m => m.Name).Returns("ModuleA");
        moduleA.Setup(m => m.ConfigureServices(It.IsAny<IServiceCollection>()))
            .Callback(() => loadOrder.Add("ModuleA"));

        // ModuleA depende de ModuleB
        moduleA.Setup(m => m.GetType().GetCustomAttribute<ModuleAttribute>())
            .Returns(new ModuleAttribute { Dependencies = new[] { "ModuleB" } });
    }
}

```

```
// Act
// ... executar factory

// Assert
Assert.Equal("ModuleB", loadOrder[0]);
Assert.Equal("ModuleA", loadOrder[1]);
}
```

PARTE 3: CHECKLIST DE CODE REVIEW

Checklist Geral (use para todos os exercícios)

Design e Arquitetura:

- ☐ Factory tem responsabilidade única e bem definida?
- ☐ Interfaces são claras e coesas?
- ☐ Não há violação de princípios SOLID?
- ☐ Padrão escolhido é apropriado para o problema?
- ☐ Código está extensível sem modificação (OCP)?

Implementação:

- ☐ Validação de entrada adequada?
- ☐ Exceções específicas e informativas?
- ☐ Factory nunca retorna null?
- ☐ Não há lógica de negócio no factory?
- ☐ Factory é stateless ou thread-safe?

Integração DI:

- ☐ Lifetimes corretos (Singleton/Scoped/Transient)?
- ☐ Não usa Service Locator anti-pattern?
- ☐ Dependências injetadas via construtor?
- ☐ Uso de IServiceProvider justificado?

Testes:

- ☐ Testes cobrem casos de sucesso?
- ☐ Testes cobrem casos de erro?
- ☐ Testes verificam integração com DI?
- ☐ Testes são independentes e determinísticos?

☐ Nomes de testes são descritivos?

Documentação:

☐ Comentários explicam "por quê", não "o quê"?

☐ Tipos suportados documentados?

☐ README com instruções de execução?

☐ Exemplos de uso presentes?

Qualidade de Código:

☐ Nomenclatura clara e consistente?

☐ Formatação adequada?

☐ Sem código comentado ou debug?

☐ Sem magic numbers ou strings?

PARTE 4: COMPARAÇÕES ERRADO vs CORRETO

Exemplo 1: Factory com Acoplamento

✖ ERRADO:

```
csharp

public class BadFactory
{
    public IPprocessor Create(string type)
    {
        // Instanciação direta = acoplamento forte
        return type switch
        {
            "A" => new ProcessorA("hardcoded-config"),
            "B" => new ProcessorB(new HardcodedDependency()),
            _ => null // Retornar null é ruim!
        };
    }
}
```

✔ CORRETO:

```
csharp
```

```

public class GoodFactory : IFactory
{
    private readonly IServiceProvider _provider;

    public GoodFactory(IServiceProvider provider)
    {
        _provider = provider;
    }

    public IProcessor Create(string type)
    {
        // Resolve via DI = baixo acoplamento
        return type.ToLower() switch
        {
            "a" => _provider.GetRequiredService<ProcessorA>(),
            "b" => _provider.GetRequiredService<ProcessorB>(),
            _ => throw new NotSupportedException($"Tipo inválido: {type}")
        };
    }
}

```

Exemplo 2: Factory Method sem Template

✗ ERRADO:

csharp

```
public abstract class BaseService
{
    protected abstract IProcessor CreateProcessor();

    // Cada subclasse duplica essa lógica
}

public class ConcreteServiceA : BaseService
{
    protected override IProcessor CreateProcessor()
    {
        return new ProcessorA();
    }

    public void Execute()
    {
        var processor = CreateProcessor();
        // Validação duplicada
        // Logging duplicado
        processor.Process();
        // Cleanup duplicado
    }
}
```



CORRETO:

csharp

```

public abstract class GoodService
{
    protected abstract IProcessor CreateProcessor();

    // Template Method reutiliza lógica comum
    public void Execute()
    {
        Validate();
        Log("Iniciando processamento");

        var processor = CreateProcessor();
        processor.Process();

        Log("Processamento concluído");
        Cleanup();
    }

    private void Validate() { /* validação comum */ }
    private void Log(string msg) { /* logging comum */ }
    private void Cleanup() { /* cleanup comum */ }
}

```

Exemplo 3: Abstract Factory Inconsistente

✗ ERRADO:

```

csharp

// Factory cria produtos de famílias diferentes!
public class InconsistentFactory : IFactory
{
    public IHeader CreateHeader() => new PdfHeader();
    public ITable CreateTable() => new ExcelTable(); // Família diferente!
    public IChart CreateChart() => new HtmlChart(); // Outra família!
}

```

✓ CORRETO:

```

csharp

```

```
// Factory cria produtos consistentes da mesma família
```

```
public class PdfFactory : IFactory
{
    public IHeader CreateHeader() => new PdfHeader();
    public ITable CreateTable() => new PdfTable();
    public IChart CreateChart() => new PdfChart();
}

public class ExcelFactory : IFactory
{
    public IHeader CreateHeader() => new ExcelHeader();
    public ITable CreateTable() => new ExcelTable();
    public IChart CreateChart() => new ExcelChart();
}
```

PARTE 5: RUBRICAS DETALHADAS

Rubrica para Exercício Básico (100 pontos)

Excelente (90-100):

- Todos os requisitos implementados corretamente
- Código limpo, bem estruturado e comentado
- Testes abrangentes com casos de borda
- Integração DI perfeita com lifetimes adequados
- Tratamento robusto de erros
- Documentação clara

Bom (75-89):

- Requisitos principais implementados
- Código funcional mas pode melhorar estrutura
- Testes cobrem casos principais
- Integração DI funciona mas lifetimes podem melhorar
- Tratamento básico de erros
- Documentação presente

Satisfatório (60-74):

- Requisitos mínimos atendidos
- Código funciona mas tem problemas de design
- Poucos testes ou testes incompletos
- Integração DI básica
- Tratamento de erros limitado
- Documentação mínima

Insatisfatório (<60):

- Requisitos não atendidos completamente
- Código com bugs ou problemas graves
- Testes ausentes ou inadequados
- Sem integração DI ou implementação incorreta
- Sem tratamento de erros
- Sem documentação

Rubrica para Exercício Avançado (100 pontos)

Excelente (90-100):

- Arquitetura modular exemplar
- Resolução de dependências perfeita
- Extensibilidade clara
- 20+ testes de alta qualidade
- Documentação completa com diagramas
- Apresentação profissional e clara defesa

Bom (75-89):

- Arquitetura modular funcional
- Dependências resolvidas corretamente
- Bom nível de extensibilidade
- 15+ testes adequados
- Boa documentação
- Apresentação competente

Satisfatório (60-74):

- Módulos funcionam mas arquitetura pode melhorar
- Dependências resolvidas mas com problemas menores
- Extensibilidade limitada
- 10+ testes básicos
- Documentação básica
- Apresentação aceitável

Insatisfatório (<60):

- Arquitetura confusa ou incorreta
 - Problemas graves na resolução de dependências
 - Não extensível
 - Poucos testes ou inadequados
 - Documentação insuficiente
 - Apresentação fraca
-

PARTE 6: CRITÉRIOS DE CORREÇÃO AUTOMATIZADA

Use estes critérios para verificação rápida:

Compilação (eliminatório):

```
bash
dotnet build
# Deve compilar sem erros
```

Testes (peso 20%):

```
bash
dotnet test
# Mínimo: 80% de cobertura nos exercícios intermediário/avançado
```

Análise Estática (peso 10%):

```
bash
```

```
dotnet format --verify-no-changes
```

```
# Código deve estar formatado corretamente
```

Métricas de Qualidade:

- Complexidade ciclomática < 10 por método
 - Linhas por método < 50
 - Classes com responsabilidade única
 - Acoplamento baixo (< 5 dependências por classe)
-

ENTREGÁVEIS ESPERADOS

Para cada exercício, o aluno deve entregar:

1. Código fonte:

- Projeto .NET compilável
- Testes unitários e de integração
- README com instruções

2. Documentação:

- Diagramas UML (classes e sequência mínimo)
- Decisões de design justificadas
- Exemplos de uso

3. Para exercício avançado, adicionar:

- Apresentação em slides (PowerPoint/PDF)
- Vídeo de demonstração (opcional, 5-10 min)
- Documento de reflexão sobre aprendizado

Formato de entrega: Repositório Git ou arquivo ZIP **Prazo:** Definido pelo instrutor para cada exercício